

Natural Language Search over Enzymes and Reactions

Jonathan Whitaker

September 17, 2026

Abstract

Contrastive models trained with large datasets of reaction–enzyme pairs are a promising approach for finding proteins with a desired function. These models typically require precise, structured inputs describing the chemical reaction of interest. In this work, we train a natural language encoder that maps plain text into a shared reaction–enzyme embedding space, enabling rapid exploration and search with informal descriptions as input. By leveraging existing language embedding models, we can train these adapters with minimal training data and compute. We show a scaling trend, where larger embedding models lead to better retrieval of relevant proteins for a given text query, with an 8-billion-parameter embedding model + adapter retrieving a relevant enzyme within the top 100 hits for approximately 69% of test reactions.

1 Introduction

Horizyn 1 is a dual-encoder contrastive model trained to embed enzymes and reactions into a shared 512-dimensional space, with applications for enzyme identification, annotation, promiscuity analysis and more. Proteins are embedded with the ProtT5 protein language model, while chemical reactions use a custom approach combining Structural & Extended-Connectivity Fingerprints (ECFP) with Differential Reaction Fingerprints (DRFP). This works, but requires a precise specification of the reaction in the correct format. While modern LLMs are decent at expressing reactions in this format, it would still be convenient if we had a model that could take in text directly, including less formal queries. In this note, we share a proof-of-concept implementation of this idea, creating an adapter that maps text embeddings into the same shared horizyn embedding space. In addition to enabling rapid exploration, this also raises some interesting questions around the future of reaction representations.

A Retrieve reactions and proteins

“making cinnamaldehyde yum yum”

↓ Cosine retrieval

REACTION Rank 1	0.359
RHEA 10392	
(E)-cinnamyl alcohol + NADP ⁺ → (E)-cinnamaldehyde + NADPH + H ⁺	
PROTEIN Rank 1	0.360
Geraniol dehydrogenase 1	
Q2KNL6 <i>Ocimum basilicum</i> (sweet basil) Reported activity on cinnamyl alcohol	

B Probe possible functions

	Lipase Q93MW7	Acetyltransferase Q9QXS4
1 “This snips apart an ester with a little help from water.”	0.367	−0.042
2 “Snips fatty acid tails off a lipid”	0.345	−0.051
3 “Nah buddy, this adds an acetyl group to a molecule.”	−0.006	0.313

Figure 1: Informal language queries can be used to retrieve reactions and proteins, or probe a protein’s possible functions

Notably, we can train these adapters on top of existing text embedding models, using relatively few examples for the training stage. By testing out embedding models of different sizes, we observe a scaling trend where larger embedding models perform better at reaction- and protein-retrieval tasks, suggesting that larger LLMs may learn more useful and nuanced internal representations of chemistry.

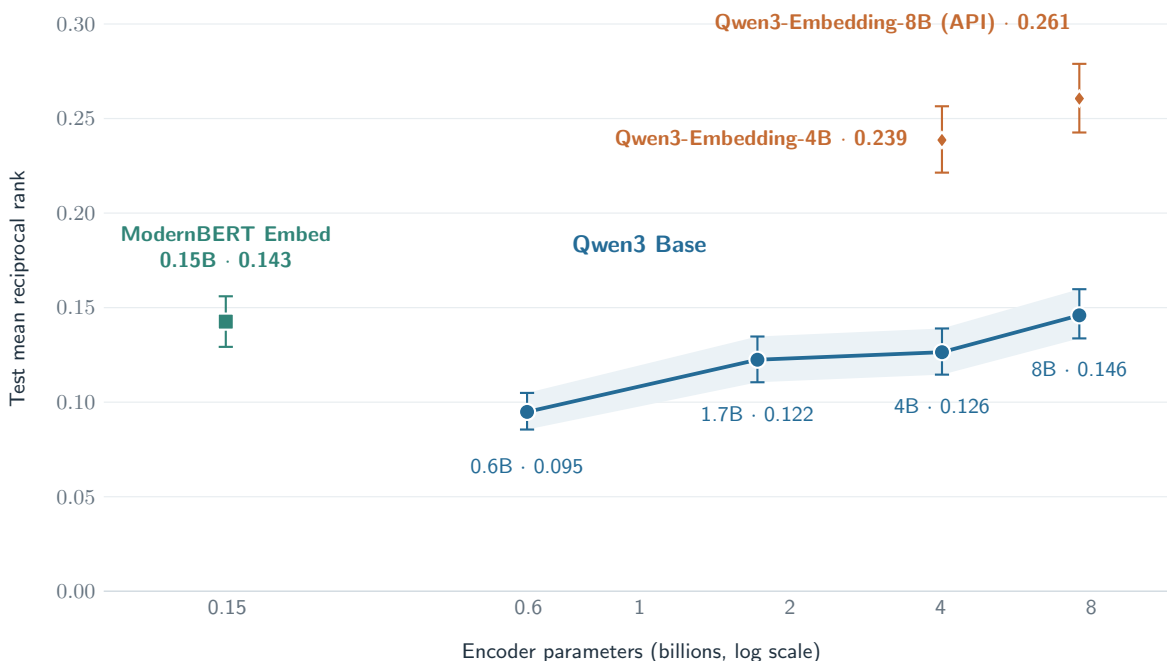
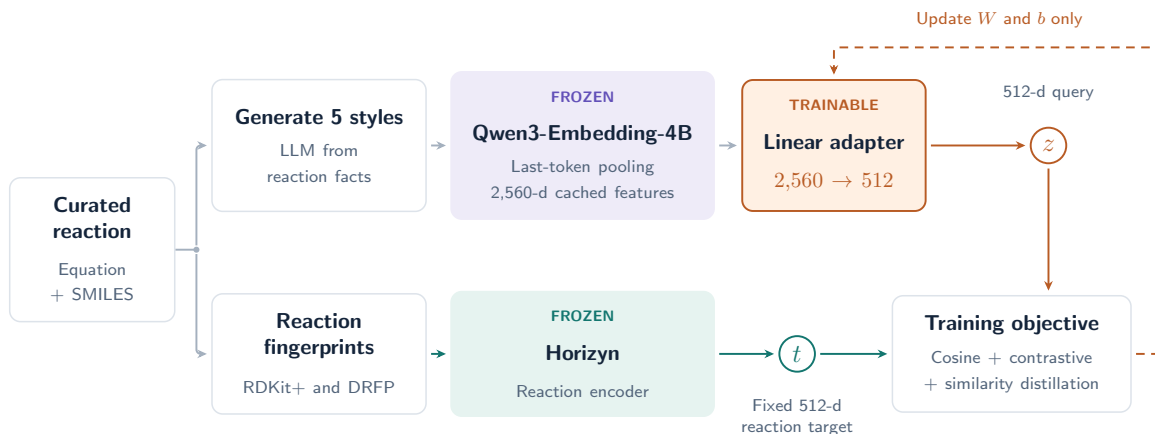


Figure 2: **Scaling text-to-reaction retrieval.** Big models have more useful embeddings, assigning lower ranks to the correct reaction in a reaction-retrieval task. MRR = Mean Reciprocal Rank, higher is better. Dedicated embedding models outperform base LLMs. Error bars show 95% reaction-group bootstrap intervals. NB: Scaling experiments used a slightly simplified training recipe.

We speculate that pushing to even more advanced reasoning models tasked with hypothesizing on the mechanisms involved may yield even more useful representations, but leave that for future work.

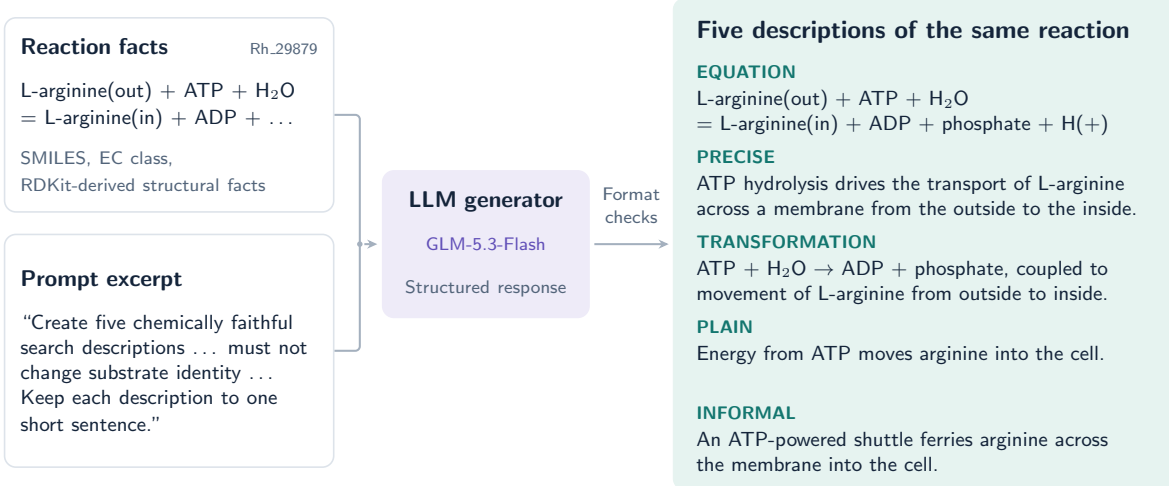
This paper will outline the approach taken, and share early results. Do not be fooled by the appearance of rigor, it is a one-person hack job inspired by a few days of vibe research! The paper itself is written with good old-fashioned human fingers, but the figures, latex typesetting and code are all done by the ever-helpful GPT-6-Astra, which deserves credit for getting all of this working so quickly. A demo is also available at <https://horizynadapter.johnnowhitaker.com/> :) Now, how do we make this magical model?

2 Approach



2.1 Training Data Generation

To generate paired data for training, we feed reactions to an LLM which is prompted to rephrase them in five different styles: ["equation", "precise", "transformation", "plain", "informal"]. For this pilot study, we used the model "z-ai/glm-5.3-flash" via OpenRouter as a cheap and fast way to get going, but we hypothesize that leveraging better models and focusing more on reasoning about the mechanisms and reaction intricacies could give a boost in terms of data quality for this stage.



Additional experiments leveraged existing natural language annotations in swissprot/uniprot, although these aren't as stylistically diverse, and many tend to be generic rather than specific to individual proteins, so their utility for fine-grained protein search may be limited.

2.2 Training The Adapters

Given a batch of 2560-dimensional Qwen embeddings X , we project to 512-d with trainable W and b :

$$u_i = \text{normalize}(x_i - \mu)$$

$$h_i = Wu_i + b$$

$$z_i = \text{normalize}(h_i)$$

Here μ is the mean text embedding across all the training description rows, calculated once. For intuition: without subtracting this mean, the dominant embedding direction would likely be a generic "looks like chemistry" direction, losing the nuance.

The resulting z is used to calculate three different loss components, which are then combined together:

$$L = L_{\text{cosine}} + 0.5L_{\text{contrastive}} + L_{\text{distill}}$$

NB: Ablations showed small gains from this multi-part loss function. Cosine similarity alone was a poor loss function, but as long as some sort of contrastive loss or distillation was used the performance roughly matched the full recipe. The scaling tests used cosine loss plus 0.15 times an in-batch contrastive loss, hence the slightly higher final scores quoted elsewhere.

Let's look at the three components individually.

1. Cosine similarity: because z_i and t_i are unit vectors, their dot product is their cosine similarity:

$$c_i = z_i^\top t_i$$

The loss is framed as follows, such that higher similarity between z_i and t_i gives a lower loss. Note: this component of the loss only considers the positive 'correct' reaction t_i .

$$L_{\text{cosine}} = \frac{1}{B} \sum_{i=1}^B (1 - c_i)$$

2. Contrastive retrieval. This compares each projected text vector with the full reaction library:

$$s_{ij} = z_i^\top R_j$$

This produces a $B \times N$ similarity matrix. Each row is converted into a probability distribution using temperature $\tau_c = 0.07$ (to emphasize score differences):

$$p_{ij}^{(c)} = \frac{\exp(s_{ij}/0.07)}{\sum_{k \in C} \exp(s_{ik}/0.07)}$$

There can be multiple correct candidates. Let P_i contain all candidates belonging to the same equivalent-reaction group as the paired target. The per-example loss is:

$$\ell_{\text{contrastive},i} = -\log \left(\sum_{j \in P_i} p_{ij}^{(c)} \right)$$

Intuition for this component of the loss: we want the high scores allocated to the correct candidates - by massaging the scores into a probability distribution that sums to 1, then adding up the entries for correct positions, we reward high scores for correct answers AND low scores for the rest.

3. Similarity distillation. The existing pre-trained horizyn reaction encoder acts as the teacher, and we train the student to assign 'similar similarities' by comparing the two.

First calculate the teacher distribution:

$$p_{ij}^{(T)} = \text{softmax}_j \left(\frac{t_i^\top R_j}{0.05} \right)$$

Then calculate the student distribution from the projected text, using the same temperature:

$$p_{ij}^{(S)} = \text{softmax}_j \left(\frac{z_i^\top R_j}{0.05} \right)$$

Then the loss is the KL divergence between these two distributions:

$$L_{\text{distill}} = \frac{1}{B} \sum_i \sum_{j=1}^N p_{ij}^{(T)} \log \frac{p_{ij}^{(T)}}{p_{ij}^{(S)}}$$

The intuition here is that we're trying to get the text encoder to match the predictions of the reaction encoder, but doing so by comparing their scores across the batch and penalizing differences between the resulting distributions.

For those who prefer code, here is a 'pytorch' version of the loss calculations described above:

```

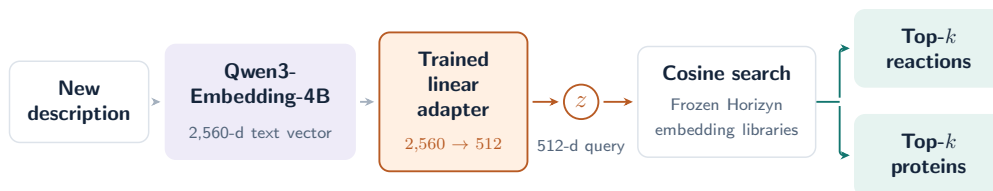
1 target = library[gold[ids]]
2
3 loss_cosine = (1 - (z * target).sum(-1)).mean()
4
5 # This is a numerically stable version of the contrastive loss described above
6 loss_contrast = (torch.logsumexp(logits, 1) - torch.logsumexp(logits.masked_fill(~
    positive, -float("inf")), 1)).mean()
7
8 teacher_dist = (target @ library.T / 0.05).softmax(1)
9 student_dist = (z @ library.T / 0.05).log_softmax(1)
10 loss_distill = F.kl_div(student_dist, teacher_dist, reduction="batchmean",)
11
12 loss = loss_cosine + 0.5 * loss_contrastive + loss_distill

```

The adapter is trained with the AdamW optimizer, using a batch size of 256. The training dataset is made up of 45,000 descriptions of 9000 reactions. Separate sets of 900 reactions (4,500 descriptions) are used as evaluation and test sets, with clustering used to reduce overlap between the train, eval and test sets. Exact info is in the appendix for those curious.

2.3 Retrieval and Evaluation

Given a trained adapter, new queries can be embedded and used to retrieve the top-matching reactions and proteins:



By using generated descriptions of the test set of reactions, we can quantify how well the model does at retrieving relevant results. The Horizyn 1 model claims to "recover an enzyme with correct activity within the top 100 hits for over 75% of test reactions". Training an adapter on top of Qwen-Embedding-8B with the full recipe results in a **67% protein R@100**. The 8B catalogue reaches 75.63%, and **true Horizyn queries reach 78.71% for this test setup**. not bad given that we're searching with natural language using an adapter trained on tens of thousands of text-reaction pairs rather than the original model's millions of reaction-enzyme pairs!

Model	Size (B)	Reactions			Proteins	
		MRR	R@10 (%)	R@100 (%)	R@10 (%)	R@100 (%)
ModernBERT-embed-base	0.149	0.1426	30.20	70.42	20.55	49.35
Qwen3-0.6B-Base	0.596	0.0949	21.82	62.31	16.31	43.83
Qwen3-1.7B-Base	1.721	0.1224	26.74	69.08	19.61	49.08
Qwen3-4B-Base	4.022	0.1264	28.11	69.98	20.55	50.03
Qwen3-8B-Base	7.568	0.1459	32.49	73.82	22.61	52.26
Qwen3-Embedding-4B	4.022	0.2386	48.68	84.51	30.39	60.74
Qwen3-Embedding-8B	7.567	0.2606	53.49	86.70	32.60	61.66

Bigger seems to be better w.r.t. embedding models, and dedicated embedding models do better than base LLMs (although those work too). NB: Scaling experiments shown in this table used an early, simplified training recipe.

For the Qwen3 Base series, the models were used as 'embedding models' by encoding the reaction in the prompt, and then treating the output hidden states for the final token (typically ".") as the 'embedding vector'. It is likely that training the last few layers with a new embedding projection head and a standardized format would yield additional gains!

The different description styles gave slightly different scores, as you might expect. For example, for the 4B embedding model in the table above, protein R@100 ranged from 58% for 'informal' descriptions to nearly 63% with the 'precise' ones.

2.4 Additional Things We Tried

There were a few extra experiments that don’t quite fit with the main thread of this paper but are worth a brief mention:

- Training on Swiss-Prot descriptions, a new adapter finds a protein sharing a known reaction in the top 10 62.0% of the time, versus 27.6% for the old adapter. However, it does far worse at reaction retrieval, and requires you to write Swiss-Prot-Style descriptions (no synthetic data generation step was used). It seems likely that, with a bit of effort on the synthetic data side, one could train a better text- $\rightarrow$ protein model rather than focusing on aligning with the reaction side as we did here in this paper.
- A ‘catalog’ approach, where a text query is compared to text embeddings of the canonical reaction strings, and then the top few reaction hits from that are used as the horizyn query, results in pretty great performance*. So rather than having input text map into horizyn space directly, simply using an off-the-shelf embedding model to retrieve related reactions and then searching with those might be a way to get most of the benefits of this paper - albeit without all the fun.
- I did try a few different ways of turning the base LLMs into embedding models, none beat the ‘dumb’ approach first proposed by Astra :shrug:
- I also tried a few API-only embedding models, such as OpenAI’s old text-embedding-large, or gemini-embedding-2. Mixed results, some did worse than even the 4B Qwen.
- Likewise for adapters, we tried fancier architectures but at the end normalization + a simple linear layer was hard to beat and made life nice and simple. This may change with scale.

*: The downside of the catalog-based approach is that it works for reactions in the catalog - we are (presumably) giving up some generalization. But within the known reactions, the catalog-based search with a 4B model scores 75.51% protein R@100, versus 60.74% for the 4B scaling-study adapter using the same encoder, and 78.71% for the true reaction input. Its plain/informal result is 72.44%. So this is a pretty strong baseline if all you want is natural language search within the existing reaction catalog.

2.5 Speculation, Limitations and Future Work

Since we’re just aligning the embedding spaces of existing encoders, this required relatively little training (1 minute on an A100 with cached embeddings). It’s impressive that it works as well as it does! However, a big motivation for me beginning to explore this route is a dissatisfaction with the existing options for encoding chemical reactions. If we’re doing our best to match a (suboptimal) existing encoding method, are we really adding much besides the convenience of text input?

I think the potential for LLM-based reaction embeddings goes much further than this. To predict the next tokens in a reaction string, LLMs must learn a lot about chemistry. And reasoning models with tools and search can augment their internal knowledge and speculate on nuanced reaction mechanics, with a breadth that no human can match. If we can leverage these learned representations properly, I suspect we can end up with a very powerful way to turn chemistry into features for embedding, search, and other ML tasks. We’ll also (potentially) get more interesting generalization - since the pre-training corpus is so vast, it is likely that modern LLMs have trained on a very diverse set of reactions along with their descriptions and interpretations; the hope is that this would mean usefulness beyond the relatively small dataset where we have paired enzymes and reactions to train on.

All that to say: I think there is lots left here to explore, mining for data, scaling up models, fishing for more paired data, and exploring additional ways to source or generate alternate representations.

2.6 AI Disclaimer

As mentioned in the intro, GPT-6-Astra was used heavily for the code implementation, although I provided plenty of input, direction and validation. The ‘vibe coded’ repo is available [on github](#) if you want to look at the code for some reason, or point your own agent at it. I can’t vouch for it all - but I can and do vouch for the contents of this paper, no slop here, and I’ve made sure, to the best of my

abilities, that the important bits of code + results are correct. Feel free to reach out with questions, corrections or suggestions. Thanks for reading, yours, J.

PS: I should cite some things! This work builds on the shared reaction-protein embedding space of Horizyn-1 [RTR⁺26], text representations from Qwen3 Embedding [ZLL⁺25] and ModernBERT model [WCC⁺25]. Protein annotations, reaction definitions, and structural visualizations draw on UniProtKB/Swiss-Prot [The25], Rhea [BMA⁺22], and the AlphaFold Protein Structure Database [BTM⁺26], respectively.

PPS: An AI-written appendix with minutiae, additional scores, and extra info can be found separately, along with the code etc, in the [accompanying GitHub repository](#).

References

- [BMA⁺22] Parit Bansal, Anne Morgat, Kristian B. Axelsen, Venkatesh Muthukrishnan, Elisabeth Coudert, Lucila Aimó, Nevila Hyka-Nouspikel, Elisabeth Gasteiger, Arnaud Kerhornou, Teresa Batista Neto, Monica Pozzato, Marie-Claude Blatter, Alex Ignatchenko, Nicole Redaschi, and Alan Bridge. Rhea, the reaction knowledgebase in 2022. *Nucleic Acids Research*, 50(D1):D693–D700, 2022.
- [BTM⁺26] Damian Bertoni, Maxim Tsenkov, Paulyana Magana, Sreenath Nair, Ivanna Pidruchna, Marcelo Querino Lima Afonso, Adam Midlik, Urmila Paramval, Dare Lawal, Ahsan Tanweer, Meera Last, Risha Patel, Agata Laydon, Dariusz Lasecki, Nick Dietrich, Hamish Tomlinson, Augustin Židek, Tim Green, Oleg Kovalevskiy, Andy Lau, Shaun Kandathil, Nicola Bordin, Ian Sillitoe, Milot Mirdita, David Jones, Christine Orengo, Martin Steinegger, Jennifer R. Fleming, and Sameer Velankar. AlphaFold Protein Structure Database 2025: a redesigned interface and updated structural coverage. *Nucleic Acids Research*, 54(D1):D358–D362, 2026.
- [RTR⁺26] Jason W. Rocks, Dat P. Truong, Dmitriy Rappoport, Samuel Maddrell-Mander, Daniel A. Martin-Alarcon, Toni M. Lee, Steven Crossan, and Joshua E. Goldford. Dual-encoder contrastive learning accelerates enzyme discovery. *Proceedings of the National Academy of Sciences*, 123(12):e2520070123, 2026.
- [The25] The UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Research*, 53(D1):D609–D617, 2025.
- [WCC⁺25] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Griffin Thomas Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2526–2547. Association for Computational Linguistics, 2025.
- [ZLL⁺25] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 Embedding: Advancing text embedding and reranking through foundation models. arXiv preprint arXiv:2506.05176, 2025.